

Hyperpure Vendor Integration

How Purchase Orders are sent to Hyperpure (Zomato's food-supply arm) for outlet-mapped GST entities, how vendor status updates come back by webhook, and how Hyperpure's own vendor pricing is synced onto the native vendor pricelist.

Module harleys_connect (application/purchase)

Status Working for order placement and webhook receipt; price sync and go-live IP whitelisting are open (see §9)

Scope res.partner / purchase.order / connect.partner / connect.endpoint / connect.manager

CONTENTS

1. The problem
2. Architecture: a generic engine, Purchase-specific logic
3. GST multi-entity vendor structure
4. Part 1 — Sending a PO (E-Transfer)
5. Part 2 — Syncing vendor prices
6. Part 3 — Receiving status updates (webhook)
7. Walkthrough (as tested)
8. Module reference
9. Open blockers: IP whitelisting and search_products
10. Design decisions and alternatives ruled out
11. Known limitations / open items
12. Full schema reference

1. The problem

Harleys buys a large share of its raw materials from Hyperpure. Historically this meant a Purchase Order approved in Odoo, then re-keyed by hand into Hyperpure's own ordering system — no shared source of truth, no automatic status feedback, and pricing that could silently drift between the two systems.

The goal: an approved Odoo Purchase Order should be submittable to Hyperpure directly, its vendor-side progress (dispatched, delivered, cancelled, partially fulfilled) should flow back into the same PO automatically, and Hyperpure's own agreed pricing should be periodically pulled into Odoo's native vendor pricelist rather than re-typed.

2. Architecture: a generic engine, Purchase-specific logic

`harleys_connect` is a vendor-agnostic external-partner integration engine — nothing Hyperpure-specific lives in its `models/` folder. Rista/Posist (a separate reporting-only integration, module `harleys_pos_registry`) is plugged into the same engine.

MODEL	ROLE
<code>connect.partner</code>	One external vendor account (credentials, auth mode, environment variables). Hyperpure is one record (id 5) shared across four vendor-facing <code>res.partner</code> entities — see §3.
<code>connect.endpoint</code>	One HTTP operation on that partner: path template, HTTP method, params (query/header/path/body, each sourced from a partner variable, a reference dataset, a static value, or the request payload), direction (push/pull), and whether it's a webhook receiver.
<code>connect.manager</code>	The single call site for every outbound <code>requests.request()</code> and every inbound <code>receive_webhook()</code> — logging (<code>connect.log</code>), retries, idempotency.
<code>processors.py</code> (<code>BaseProcessor</code> , <code>ProcessorRegistry</code> , <code>ProcessorResult</code>)	Vendor-agnostic response handling contract. Purchase registers its own processors (§6) without <code>harleys_connect</code> itself knowing anything about purchase orders.

Everything Hyperpure/Purchase-specific — the payload shape, the price-sync wizard, the webhook status processor, the delivery-slot field — lives under `harleys_connect/application/purchase/`, resolving handlers from vendor bindings *at execution time*, not at registry startup.

3. GST multi-entity vendor structure

Hyperpure requires GST-registration-level separation: a single API account (one `connect.partner`) is split across **four** `res.partner` vendor records, one per GST entity/state. `product.supplierinfo` is already naturally segmented per vendor record — no separate product catalogue needed per GST, each entity just has its own price/vendor-code rows against the shared product list.

ENTITY	RES.PARTNER ID	GSTIN	OUTLET DATASET	PRODUCT CODES MAPPED
HYD / Telangana	31	36AAACZ8867B1Z1	<code>outlets</code> — 2 real Hyperpure outlets	5 of 134 supplierinfo rows
BLR / Karnataka	82	29AAACZ8867B1ZW	<code>outlets_blr</code> — 25 outlets, placeholder ids	0 of 156
MH / Maharashtra	83	27AAACZ8867B1Z0	<code>outlets_mh</code> — 22 outlets, placeholder ids	0 of 246
NCR / Haryana	5387	06AAACZ8867B1Z4	<code>outlets_ncr</code> — 7 outlets, placeholder ids	0 of 86

Only HYD has real Hyperpure-issued outlet ids (`802249` "HYD-CS", `802297` "HYD-KMP" — both labelled `EX-HP-HARLEYS-TEST*` on Hyperpure's own buyer portal). BLR/MH/NCR's outlet ids (`9000001`+) are internal placeholders pending real values from Hyperpure.

UNMAPPED WAREHOUSES

`VIJ` (7 warehouses) and `WH` (1 warehouse) are not yet mapped to any of the four GST entities — unresolved, needs user/Hyperpure input before those warehouses can transact with Hyperpure at all.

4. Part 1 — Sending a PO (E-Transfer)

Entry point: `action_send_to_vendor()` on `purchase.order`, exposed as a button (`show_vendor_send_button`, visible once a PO is approved and its vendor has a Connect integration). It filters out non-integrated vendors silently (a broad list selection routinely includes ordinary vendors too) and opens `connect.order.confirm.wizard` in mode `transfer` — the same wizard model used for price sync (§5), reused rather than duplicated.

Pre-flight checks — `_check_transfer()`

Before any network call: PO is `state == 'purchase'`, not already sent/mid-submission, the vendor connection is authenticated, vendor units are confirmed (`connect_units_confirmed`), a resolvable vendor currency exists and matches the PO's own currency, Validate/Place endpoints (if configured) are active Push endpoints with a generated handler reference and identical static body fields, and `Idempotent Safe Retry` is disabled on placement (a retried "safe" call would silently duplicate a real vendor order).

Building the payload — `_build_hyperpure_payload()`

Per order line: resolves the vendor product code and quantity factor via `vendor._item()` (§5 covers where that code comes from), converts to whole vendor sub-units (rejects non-integer quantities), and includes `reference_price` (tax-excluded line subtotal per sub-unit) + `is_strict_pricing: true` unless the code is in an explicit omit-set (used when re-submitting after accepting a vendor price mismatch). Two fields are deliberately **never** sent unless explicitly set:

- `delivery_date` — Hyperpure computes its own Target Delivery Date per outlet, confirmed live to shift over time (advanced a full day between two submissions minutes apart); a guessed date that doesn't match gets the whole order rejected, so it's omitted entirely.
- `delivery_slot` (`connect_delivery_slot` field on the PO) — same rejection risk per Hyperpure's own docs if it doesn't match their actual assigned slot, so it's opt-in per order rather than derived.

A DOCUMENTED TYPE CONTRADICTION, RESOLVED BY TRUSTING THE SAMPLES

`product_number`: Hyperpure's own field-type table says integer, but every sample request body in their docs sends it quoted as a string. The code sends it as a string (`str(int(code_number))`) — matches the samples, which turned out to be what their API actually accepts, not the schema table.

Submitting — `_transfer_isolated()`

Runs on an **owned cursor** (not the caller's), row-locks the PO (`SELECT ... FOR UPDATE`) before every state transition, and commits a "submission fence" (`vendor_transfer_state = 'submitting'`) before calling Place — a worker crash after that point leaves the PO blocked for manual reconciliation rather than silently eligible for accidental duplication on retry. Validate (optional per vendor) runs first if configured; a rejection there never touches "submitting" state. Placement's own response carries only an acknowledgement message — no vendor order number yet, since that arrives later by webhook (§6).

Humanizing vendor errors — `_humanize_error()` / `_extract_price_corrections()`

A raw "HTTP 400: validation failed for some products in the order" tells a buyer nothing actionable. When Hyperpure's response includes entity-level detail (product name + plain-language reason, e.g. *"RE PAPER STRAWS 6MM: Price difference between actual price (₹ 97.00) and price entered by user (₹ 101.00) is less than lower threshold..."*), that's shown instead. `_extract_price_corrections()` separately pulls out just the `PRODUCT_PRICE_MISMATCH` rows as a code → expected-price map, so the review wizard can offer "accept vendor pricing" per line rather than forcing a full manual re-key.

5. Part 2 — Syncing vendor prices

`action_sync_vendor_prices()` on the vendor (Purchase-manager gated) opens the same `connect.order.confirm.wizard` in mode `prices`.

Where the vendor product code comes from — `_item()`

Two mutually exclusive sources, resolved per vendor:

- **Item reference dataset** (`connect_item_dataset_id`) — an uploaded CSV/XLSX, looked up by the product's Internal Reference (`default_code`). Also supplies the search keyword and quantity factor columns.
- **Native pricelist fallback** (no item dataset configured — the case for all four Hyperpure entities today) — the product's own `product.supplierinfo.product_code` for that vendor, required to be a single unambiguous integer across all matching seller rows. The search keyword falls back to the product's own `name` in this case — see the callout below.

SEARCH KEYWORD IS OUR PRODUCT NAME, NOT HYPERPURE'S CATALOGUE NAME

Since none of the four entities have an item dataset, every `search_query` sent to Hyperpure is literally `product.name` — e.g. `RE PAPER STRAWS 6MM`, `MQ BUTTER PAPER`. Those `RE` / `MQ` / `2M` prefixes are Harley's own internal SKU-family abbreviations, not Hyperpure catalogue phrasing (confirmed against a real completed sandbox order: their actual catalogue names look like *"Hyperpure - Butter Croissant, Handrolled (75 gm/pc) (Frozen)"*). This is one contributing factor in the `search_products` investigation — see §9.

Preview — `_preview_price_sync()`

Builds the product set to reprice (from the item dataset, or every product with an existing supplierinfo row for this vendor — capped at 1–500 products per batch), then per product calls `vendor._price(product, warehouse, live=True, ...)` — a real Hyperpure `search_products` lookup by vendor code — for `vendor_price`, and separately `live=False` for `our_price` (today's native-pricelist price, for the side-by-side comparison only). Lines that fail the live lookup keep `vendor_price = 0.0` and get a `mismatch_note`; they are not treated as errors that block the rest of the batch.

Apply — `action_apply_price_sync()`

Per successfully-synced line: finds the matching `product.supplierinfo` row (variant match first, template-level fallback), skips (and reports) ambiguous matches — e.g. multiple quantity tiers — rather than guessing. Row-locks and re-checks the existing price against what was shown at preview time (`preview_context` fingerprint also catches outlet/endpoint/dataset drift since preview); a changed price is skipped and flagged, not silently overwritten. Writes the synced price **in place** — no history row — or creates a new supplierinfo row (stamping currency only on creation) if the product had none yet.

BUG FIXED 2026-09-23 — PARTIAL COVERAGE NO LONGER BLOCKS APPLY

`price_check_done` (gates the Apply button) used to require every previewed product to have synced successfully. With only 5 of HYD's 134 mapped products having a vendor code, the Apply button never appeared for a real batch. Changed to require only *some* line to have synced (`bool(available_count)`) — Apply now writes whatever succeeded and leaves the rest at their default zero, matching how the write logic itself already tolerated partial coverage line-by-line. The preview summary now also states the split explicitly, e.g. *"5 of 115 products have a synced price; the rest have no vendor product code yet and are left unchanged."*

This replaced an earlier CSV/reference-dataset mechanism entirely (fields

`connect_price_dataset_id` / `connect_price_code_column` / `connect_price_outlet_column` / `connect_price_value_column`

OPEN DESIGN QUESTION, UNRESOLVED

`_build_hyperpure_payload()` sends the PO line's own stored `price_unit` as `reference_price` at submission time — not a fresh pull of the currently-synced vendor price. These can drift apart (PO created, then supplierinfo re-synced later, never re-linked). A read-only "Vendor Ref. Price" column (`connect_vendor_price`, computed via `_price(live=False)`) already shows the current synced price for comparison on the PO line. Whether E-Transfer should substitute the live synced price instead of the PO's own price — trading away `is_strict_pricing`'s value as a genuine "did we agree on this" check — is not yet decided.

6. Part 3 — Receiving status updates (webhook)

Route: `controllers/webhook.py` — `POST /harleys_connect/webhook/<partner_code>/<token>/<operation>`, `auth="public"` (necessarily — Hyperpure has no Odoo user identity), `csrf=False`.

Request gating, in order

- 1. Partner lookup by `(partner_code, webhook_token, active)` — unknown or wrong token → plain 404, before anything else runs. This 128-bit random token is the **primary, and currently only active**, access gate.
- 2. Optional header-secret check (`webhook_header_name` / `webhook_header_variable_key`, compared with `hmac.compare_digest`) — **exists in code but is deliberately not yet configured** for Hyperpure, pending their real secret value. Until it is, the URL token alone is what stands between a caller and this endpoint.
- 3. Endpoint lookup (matching `operation_key`, must be an active webhook-flagged `connect.endpoint`) → 404 if not found.
- 4. Body must be JSON → 400 otherwise.
- 5. Optional `Idempotency-Key` header — a prior successful delivery with the same key short-circuits to a plain "duplicate" 200 without reprocessing.
- 6. `connect.manager.receive_webhook()` resolves the registered processor (`_get_response_processor`, matched by which vendor record has this endpoint bound as its webhook) and calls it.

SECURITY POSTURE, STATED PLAINLY

Right now, anyone who obtains the URL can call it with no further authentication. This is a reasonable baseline for controlled testing (the token is unguessable), but the header-secret check should be turned on with Hyperpure's real value before this is trusted at production scale. The blast radius of even a fully "successful" forged call is narrow either way — see below — but it's not nothing.

`PurchaseOrderUpdateProcessor` — the status-rank state machine

Each event in the payload's `orders` array correlates to a PO via `external_order_id` (must resolve to a PO already bound to this exact vendor + webhook endpoint, and with a real pending submission — `vendor_order_sent` or `vendor_transfer_state in ('submitting', 'unknown')`). Row-locked before every write.

STATUS	RANK
PLACED	1

STATUS	RANK
COMPLETED	2
DISPATCHED	3
DELIVERED	4 (terminal)
CANCELLED	4 (terminal)

A lower-ranked update than what's already stored is silently ignored (protects against out-of-order delivery); DELIVERED/CANCELLED are locked once reached — a later update for the same vendor order number is dropped unless it's the same status again. Where an order was fulfilled across multiple vendor sub-orders and they disagree, the overall status is `PARTIAL`, optionally relabelled through an admin-configured status dataset (`connect_status_dataset_id`). Every delivery — not just the field-overwriting current status — is also recorded as a chatter note (`"Vendor webhook: order %s -> %s"`), so the full history survives even though the summary field itself only ever shows the latest state. **Vendor delivery is not an Odoo stock receipt or accounting event** — this processor never touches either; Odoo's own approval/receipt/billing states are untouched.

CHATTER ALWAYS SAYS "PUBLIC USER" — THIS IS EXPECTED, NOT A BUG

`message_post()` 's default author is `self.env.user`, which resolves to Odoo's generic public user under an `auth="public"` route — true whether the real caller is Hyperpure, a test curl, or Postman. The note body's own text ("Vendor webhook: ...") is what actually identifies the source. Changing the displayed author would require explicitly overriding `author_id` on the post — a cosmetic option, not currently done.

odoo.sh production feasibility

Receiving this webhook is unaffected by odoo.sh having no static egress IP — any reverse-proxy-fronted HTTPS endpoint works. This was proven directly: an ngrok tunnel + nginx in front of the admin dev server stood in for the role odoo.sh's own front proxy would play, and real external POSTs (curl, and a Postman collection run through the collection runner) were received, processed, and reflected correctly end to end. The real blocker is **outbound** calls (Validate/Place/search_products) — see §9.

7. Walkthrough (as tested)

The exact sequence used to verify order placement and webhook receipt, run twice — once on the original local dev server, once from scratch on a second, independently re-provisioned admin/dev machine.

1 Configure the vendor's Purchase Integration



Screenshot: vendor contact form → Purchase Integration Configuration → Actions and Endpoints tab, showing the bound price/validate/place/cancel/webhook endpoints

`vendor-purchase-integration-configuration.png` — reached via the **Purchase Integration Configuration** button on the vendor's own contact form.

2 Send an approved PO to the vendor

 Screenshot: Purchase Integration Review wizard, transfer mode, price comparison table

Screenshot: Purchase Integration Review wizard (mode: E-Transfer), showing current vs. reference price per line before sending

etransfer-review-wizard.png

3 Confirm the order landed on Hyperpure's own side

 Screenshot: dev.hyperpure.com buyer portal, order detail page, matching items and total

Screenshot: Hyperpure's own dev buyer portal (dev.hyperpure.com), order detail page for the same PO, confirming items/quantities/total match

hyperpure-buyer-portal-order.png — cross-checked directly against Hyperpure's own system, not just our own success message.

4 Deliver a webhook status update and confirm the chatter

 Screenshot: Purchase Order form, chatter showing 'Vendor webhook: order ... -> DISPATCHED'

Screenshot: Purchase Order form, chatter panel showing a "Vendor webhook: order <number> -> <status>" note, posted by Public user

po-chatter-webhook-note.png — one note per delivery, and the Vendor Orders summary field updated alongside it.

5 Verify the webhook externally, outside our own infrastructure

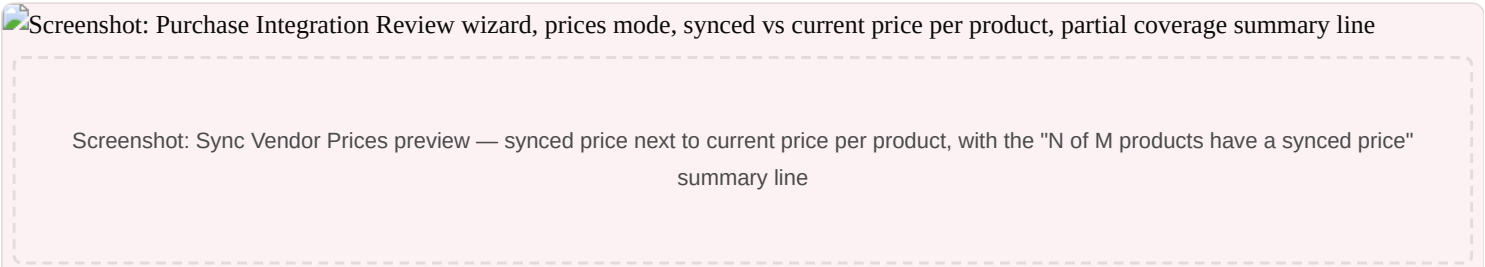
 Screenshot: Postman collection runner, 5 webhook requests (PLACED/DISPATCHED/DELIVERED/CANCELLED/unknown-PO), all passing

Screenshot: Postman collection runner results — PLACED, DISPATCHED, DELIVERED (with order_items), CANCELLED, and an unknown-PO negative case, all behaving as predicted

postman-collection-runner-results.png — run against an ngrok tunnel in front of the dev server, confirming the endpoint is reachable and behaves correctly from a genuinely external network, not just localhost.

6

Sync vendor prices



Screenshot: Sync Vendor Prices preview — synced price next to current price per product, with the "N of M products have a synced price" summary line

sync-vendor-prices-preview.png

8. Module reference

FILE	RESPONSIBILITY
<code>application/purchase/models/res_partner.py</code>	Per-vendor Connect configuration (endpoint bindings, datasets, units/currency confirmation), <code>_item()</code> / <code>_outlet()</code> / <code>_price()</code> / <code>_purchase_currency()</code> , <code>action_sync_vendor_prices()</code> , <code>action_refresh_vendor_outlets()</code> .
<code>application/purchase/models/purchase_order.py</code>	Vendor order state fields, <code>action_send_to_vendor()</code> , <code>_check_transfer()</code> , <code>_build_hyperpure_payload()</code> , <code>_transfer_isolated()</code> , <code>_humanize_error()</code> / <code>_extract_price_corrections()</code> , <code>action_cancel_vendor_order()</code> , the PO-line vendor-code/price preview fields.
<code>application/purchase/models/connect_manager.py</code>	<code>PurchaseCatalogueProcessor</code> (rejects unattended price application via a job), <code>PurchaseOrderUpdateProcessor</code> (the webhook status-rank state machine, §6), and <code>_get_response_processor()</code> — resolves which processor applies per-endpoint at execution time, not registry startup.
<code>application/purchase/models/connect_partner_outlet.py</code>	<code>connect.partner.outlet</code> — a materialized, pickable mirror of one row from the outlet reference dataset, kept in sync by <code>reference_dataset.py</code> whenever the underlying file changes. Exists so a human can pick a specific outlet from a real list rather than typing a raw external id.
<code>application/purchase/models/reference_dataset.py</code>	Keeps <code>connect.partner.outlet</code> synced whenever a vendor's outlet dataset file is re-uploaded.
<code>application/purchase/models/product_supplierinfo.py</code>	Adds <code>mail.thread</code> tracking (<code>price</code> , <code>product_code</code>) to core <code>product.supplierinfo</code> — chatter history of price changes, including ones written by Sync Vendor Prices.
<code>application/purchase/wizard/connect_order_confirm_wizard.py</code>	The single review/apply wizard, shared by both E-Transfer (<code>mode="transfer"</code>) and Sync Vendor Prices (<code>mode="prices"</code>) — <code>_prepare_review()</code> ,

FILE	RESPONSIBILITY
	<code>_preview_price_sync()</code> , <code>action_confirm_send()</code> , <code>action_apply_price_sync()</code> .
<code>application/purchase/wizard/connect_order_confirm_wizard_line.py</code>	One line per product/PO-line under review: current price, synced/reference price, mismatch flags, vendor product code.
<code>application/purchase/views/res_partner_views.xml</code>	Vendor form's "Sync Vendor Prices" quick-action button; the Purchase Integration Configuration screen (Actions and Endpoints / Reference Datasets / Order Status tabs).
<code>application/purchase/views/purchase_order_views.xml</code>	Vendor Outlet / Delivery Slot fields, Vendor Orders summary, send/cancel buttons on the PO form.
<code>application/purchase/views/vendor_pricelist_views.xml</code>	Click-to-open (non-inline-editable) override for the plain "Vendor Pricelists" menu, so inline edit stays exclusive to the "By Vendor" screen.
<code>application/purchase/views/product_supplierinfo_views.xml</code>	Adds the chatter widget to the vendor pricelist line form (for the tracking in <code>product_supplierinfo.py</code>).
<code>controllers/webhook.py</code>	The one inbound HTTP route for every vendor's webhook (\$6).
<code>models/connect_manager.py</code> (generic)	<code>execute_by_handler()</code> — the single outbound <code>requests.request()</code> call site for every push/pull operation, across every vendor.
<code>models/connect_reference_dataset.py</code>	Generic file-backed lookup table mechanism — outlet mapping, item mapping, and status-label mapping are all instances of this one model, not bespoke per-purpose tables.

9. Open blockers: IP whitelisting and search_products

IP whitelisting (blocks production go-live)

Hyperpure mandates IP whitelisting on every outbound request. odoo.sh has no static IP (confirmed via Odoo's own docs — the `/_odoo.sh/ip-change` notification mechanism exists specifically because IPs rotate). Escalated to the Odoo.sh team directly by the user; unresolved as of this writing. **Fallback if Odoo.sh can't help:** route outbound Hyperpure calls through a fixed-IP proxy — one `proxies=` change at `connect_manager.py`'s single `requests.request()` call site.

`search_products` (blocks live price lookups in both E-Transfer review and Sync Vendor Prices)

100% failure rate — every logged call since 2026-09-16 (110+ calls) returns the same generic error, for every query tried: our own internal product names, generic terms ("milk"), and real confirmed catalogue names pulled directly from an actual completed sandbox order ("Dotpe QA Testing", "Hyperpure - Butter Croissant, Handrolled (75 gm/pc) (Frozen)"). Two repeats of an identical real-name query returned *different* status codes (400, then 500) — the endpoint itself is unstable, not cleanly gated by whether the search term matches Hyperpure's catalogue.

Checked field-by-field against Hyperpure's own "HP ↔ External POS integration" API document:

- Path, method (GET), headers, and both query params (`outlet_id` , `search_query`) match the doc's field table exactly. One real gap found and fixed: `search_query` was configured `required=False` ; the doc marks it mandatory.
- The error body received — `{"error": {"code": "BAD_REQUEST_ERROR", "data": null, "message": "Something went wrong"}}` — matches the doc's own "**Generic system errors**" example verbatim (§4.8.8): schema-level rejection before field validation, no entity detail, "may ask the user to retry after some time."
- The doc's **Test Credentials** section confirms `account_id 194841` is genuinely Hyperpure's designated test account (exact match) — but states outlet/product catalogue mappings for test outlets must be **manually provisioned by Hyperpure's own tech team** on request.

CONCLUSION AND NEXT ACTION

Request construction is clean against the documented contract — nothing left to fix on our side for this endpoint. The likeliest cause is that outlet `802249` is active enough to accept real orders but was never fully provisioned with a searchable product catalogue on Hyperpure's backend. Action needed (not yet sent): email `integrations@hyperpure.com` with account 194841 + outlet 802249, confirming the request matches their own documented contract, and asking them to verify/complete catalogue provisioning for search specifically.

10. Design decisions and alternatives ruled out

- **One shared review/apply wizard for both E-Transfer and Sync Vendor Prices** — adopted over two separate wizards, since both are fundamentally "preview a price comparison, then commit" flows differing only in what gets written on Apply.
- **CSV/reference-dataset price cache (`connect.partner.reference.dataset` for prices)** — removed entirely in favour of writing synced prices directly onto `product.supplierinfo` . Kept the file-backed dataset mechanism itself (still used for outlet and item mapping) but dropped the price-specific fields (`connect_price_dataset_id`) and its column configuration) since a live-synced native pricelist row supersedes a separately cached file.
- **Sending `delivery_date` /computing a guessed delivery slot** — rejected: Hyperpure's own Target Delivery Date is authoritative and shifts over time; a mismatched guess gets the whole order rejected. Left optional and vendor/human-driven instead.
- **`validate_order_placement` pre-flight step** — deliberately skipped in favour of relying on `place_order` alone; settled, not revisited.
- **A rotating JWT Bearer token for Hyperpure auth** — an earlier assumption, corrected this session after checking live `connect.endpoint.param` config directly: Hyperpure's actual auth is `otp_token_exchange` , with static per-request headers (`ApiKey` / `ClientSecret` / `X-AccountId`), not a refreshed Bearer token.

11. Known limitations / open items

- **`search_products` outage** (§9) — blocks live price lookups everywhere they're used. Not yet reported to Hyperpure despite being flagged repeatedly.
- **IP whitelisting** (§9) — blocks all outbound calls once this runs on odoo.sh production.
- **BLR/MH/NCR have zero vendor product codes mapped** — 488 of 622 total supplierinfo rows across the four entities have no code at all, so Sync Vendor Prices produces nothing for those three entities regardless of the `search_products` outage being resolved.
- **`VIJ` / `WH` warehouses unmapped to any GST entity** — needs user/Hyperpure input.

- **Price drift, unresolved** (§5) — PO's stored price vs. the currently-synced vendor price can disagree; no decision yet on whether E-Transfer should substitute the live value.
- **Webhook header-secret check not yet enabled** (§6) — the URL token is the only active gate today; enable the header check once Hyperpure provides their real secret value.
- `share_external_order_reference_details` **must be explicitly requested from Hyperpure** during webhook setup, or `external_order_id` won't be present in their payload at all — breaks `PurchaseOrderUpdateProcessor`'s correlation entirely if missed.
- **QBP (quantity-based pricing slabs)** exists in Hyperpure's own pricing model; `search_products`'s `MarketPrice` is pre-QBP and nothing here slab-adjusts it. Flagged as a possible secondary contributor to past price-mismatch rejections, not confirmed.

12. Full schema reference

TABLE	CHANGE
<code>purchase_order</code>	+ <code>vendor_order_sent</code> , <code>vendor_order_id</code> , <code>vendor_order_number</code> , <code>vendor_order_status</code> , <code>vendor_orders</code> (JSON), <code>vendor_transfer_state</code> , <code>vendor_transfer_message</code> , <code>vendor_submission_outlet_ref</code> , <code>connect_outlet_ref</code> (FK <code>connect_partner_outlet</code>), <code>connect_delivery_slot</code>
<code>purchase_order_line</code>	computed-only: <code>connect_vendor_product_code</code> , <code>connect_vendor_price</code> (not stored)
<code>product_supplierinfo</code>	<code>mail.thread</code> added; <code>price</code> / <code>product_code</code> now <code>tracking=True</code>
<code>connect_partner_outlet</code>	new — materialized outlet-dataset rows, one per real vendor outlet, per dataset
<code>connect_order_confirm_wizard</code> / <code>_line</code>	transient — the shared review/apply wizard and its per-product/per-line rows
<code>res_partner</code>	+ <code>connect_partner_id</code> , endpoint binding fields (<code>connect_price_endpoint_id</code> , <code>connect_validate_endpoint_id</code> , <code>connect_place_endpoint_id</code> , <code>connect_cancel_endpoint_id</code> , <code>connect_webhook_endpoint_id</code>), dataset fields (<code>connect_outlet_dataset_id</code> , <code>connect_item_dataset_id</code> , <code>connect_status_dataset_id</code> + column config), <code>connect_units_confirmed</code> , <code>connect_price_currency_id</code>

Internal implementation reference — Harleys Odoo 19 Enterprise. Click any screenshot to view it enlarged and step through the rest. Looking for a non-technical walkthrough instead? See the [User Guide](#).